

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI TP. HỒ CHÍ MINH
VIỆN CÔNG NGHỆ THÔNG TIN VÀ ĐIỆN, ĐIỆN TỬ



ĐỒ ÁN MÔN HỌC
THỰC TẬP TỐT NGHIỆP

XÂY DỰNG WEBSITE QUẢN LÝ SÂN BÓNG MINI

GVHD:

Sinh viên thực hiện :

2251120252 – Võ Tấn Thịnh

TP.HCM, tháng 10, năm 2025

Mục Lục

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	2
1.1. Giới thiệu đề tài	2
1.2. Mục tiêu đề tài	2
1.3. Phạm vi nghiên cứu	4
1.4. Cơ sở lý thuyết.....	4
CHƯƠNG 2 : THIẾT KẾ HỆ THỐNG.....	6
2.1. Phân tích yêu cầu hệ thống	6
2.2. Kiến trúc hệ thống (System Architecture)	9
2.3. Thiết kế Cơ sở dữ liệu (Database Design).....	10
CHƯƠNG 3: ĐÁNH GIÁ CÔNG NGHỆ VÀ HIỆN THỰC HỆ THỐNG ..	15
3.1. Đánh giá lựa chọn Frontend - React.js	15
3.2. Đánh giá lựa chọn Backend - Node.js + Express.js	16
3.3. So sánh và lựa chọn Database (MySQL vs NoSQL)	16
3.4. Cơ chế Bảo mật và Xác thực (Authentication & Security)	17
CHƯƠNG 4: KẾ HOẠCH TRIỂN KHAI VÀ QUẢN LÝ DỰ ÁN.....	19
4.1. Phân tích hiện trạng và Tiến độ	19
4.2. Lộ trình phát triển chi tiết (Sprint Planning)	19

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu đề tài

Trong bối cảnh nhu cầu giải trí và thể thao cộng đồng tăng cao, quản lý sân bóng mini đã trở thành một lĩnh vực kinh doanh tiềm năng. Tuy nhiên, nhiều cơ sở sân bóng hiện nay vẫn sử dụng phương pháp quản lý thủ công hoặc các hệ thống lạc hậu, gây khó khăn trong việc theo dõi lịch đặt sân, quản lý thanh toán và khách hàng.

Những thách thức hiện tại bao gồm:

- Hệ thống đặt sân trực tuyến kém thân thiện, khó sử dụng trên di động.
- Không có cơ chế quản lý khách hàng và lịch sử giao dịch rõ ràng.
- Tình trạng sân không được cập nhật thực time, dễ xảy ra trùng lặp hoặc nhầm lẫn.
- Báo cáo doanh thu, thống kê hoạt động không chi tiết hoặc thiếu trực quan.

Đề tài "Xây dựng website quản lý sân bóng mini sử dụng React, Node.js và MySQL" được phát triển nhằm giải quyết các vấn đề nêu trên. Mục tiêu là tạo ra một hệ thống quản lý toàn diện, với giao diện hiện đại, dễ sử dụng và khả năng mở rộng cao, giúp chủ sân bóng quản lý hiệu quả các sân, khách hàng và doanh thu.

1.2. Mục tiêu đề tài

1.2.1. Mục tiêu nghiệp vụ

Xây dựng website quản lý sân bóng mini gồm hai phần chính:

Trang khách hàng (Client):

- Xem danh sách sân bóng có sẵn, giờ mở cửa và giá cho thuê theo thời gian.
- Tìm kiếm sân theo loại (5v5, 7v7), địa điểm, giá hoặc giờ khả dụng.
- Xem chi tiết thông tin sân, xem ảnh và quy tắc sân.
- Đặt sân trực tuyến, chọn ngày giờ, thanh toán tiền cọc.
- Quản lý các đơn đặt sân của mình, xem lịch sử ghi danh.
- Liên hệ và nhận thông báo về trạng thái đặt sân.

Trang quản trị (Admin):

- Quản lý thông tin sân bóng (thêm/sửa/xóa/xem).
- Quản lý giá, khung giờ mở cửa và tình trạng khả dụng của từng sân.
- Duyệt, xác nhận hoặc hủy các đơn đặt sân của khách hàng.

- Quản lý thông tin khách hàng, lịch sử các lần sử dụng dịch vụ.
- Xem báo cáo thống kê doanh thu, tỉ lệ sử dụng sân, khách hàng thường xuyên.
- Thiết kế giao diện hiện đại, dễ sử dụng, responsive trên mọi thiết bị.
- Đảm bảo hiệu năng và bảo mật cho toàn hệ thống.

1.2.2. Mục tiêu kỹ thuật

Frontend (React):

- Xây dựng ứng dụng SPA (Single Page Application) bằng React.
- Sử dụng React Router DOM để điều hướng giữa các trang và màn hình khác nhau.
- Gọi API qua Axios, xử lý dữ liệu bất đồng bộ và quản lý loading states.
- Quản lý trạng thái ứng dụng (thông tin người dùng, đặt sân, giỏ hàng ảo) bằng React Context API hoặc Zustand.
- Thiết kế giao diện với CSS/Bootstrap, hỗ trợ dark mode và responsive design.
- Dùng HeroIcons để tăng tính trực quan cho các biểu tượng.
- Áp dụng React Toastify để hiển thị thông báo người dùng (ví dụ: đặt sân thành công, thanh toán hoàn tất).
- Tích hợp lịch tương tác (calendar) để khách hàng dễ chọn ngày giờ đặt sân.

Backend (Node.js + Express):

- Xây dựng RESTful API xử lý dữ liệu cho frontend.
- Sử dụng MySQL2 để kết nối và truy vấn cơ sở dữ liệu MySQL.
- Tổ chức mô hình backend theo kiến trúc MVC (Model - View - Controller) giúp dễ mở rộng.
- Áp dụng JWT (jsonwebtoken) cho xác thực người dùng và quản lý phiên đăng nhập.
- Bảo mật dữ liệu người dùng bằng bcrypt để mã hóa mật khẩu.
- Sử dụng Multer để xử lý upload ảnh sân và các tài liệu liên quan.
- Dùng CORS để cho phép frontend React truy cập API từ nhiều nguồn khác nhau.
- Dùng Dotenv để quản lý biến môi trường (PORT, DB credentials, JWT secret).
- Kiểm thử API bằng Postman để đảm bảo tính đúng đắn.

Database (MySQL):

- Thiết kế cơ sở dữ liệu quan hệ gồm các bảng: users, pitches (sân), time_slots (khung giờ), bookings (đơn đặt sân), payments (thanh toán), reviews (đánh giá).
- Dùng MySQL Workbench để thiết kế và quản lý cơ sở dữ liệu.
- Kết nối từ backend qua thư viện mysql2 (Promise-based).
- Thiết lập ràng buộc khóa ngoại (foreign key) giữa các bảng để đảm bảo tính toàn vẹn dữ liệu.

Công nghệ hỗ trợ thêm:

- Nodemon, ESLint + Prettier, Git + GitHub, Vercel / Render / Railway.

1.3. Phạm vi nghiên cứu

Trong phạm vi:

- Thiết kế và xây dựng hệ thống website quản lý sân bóng mini full-stack (React + Node.js).
- Cài đặt API CRUD cho các đối tượng chính: sân bóng, khung giờ, khách hàng, đơn đặt sân, thanh toán.
- Giao diện khách hàng hiển thị danh sách sân, chi tiết, xem tính khả dụng và đặt sân.
- Giao diện quản trị viên quản lý sân, khung giờ, đơn đặt sân, khách hàng và thống kê.
- Lưu trữ dữ liệu trong cơ sở dữ liệu MySQL với các bảng liên kết hợp lý.
- Xử lý upload hình ảnh sân bóng bằng Multer.
- Bảo mật cơ bản (JWT, bcrypt, xác thực API, ủy quyền người dùng).
- Responsive design cho máy tính, tablet, điện thoại.

Ngoài phạm vi:

- Tích hợp cổng thanh toán online (VNPay, Momo, v.v.).
- Gửi email hoặc SMS xác nhận đơn đặt sân tự động.
- Hệ thống đánh giá và review nâng cao.
- Tính năng book lặp lại (booking recurring) cho các khách hàng thường xuyên.
- Dashboard thống kê nâng cao với biểu đồ doanh thu, phân tích hành vi khách hàng.

1.4. Cơ sở lý thuyết

1.4.1. ReactJS: Thư viện JavaScript mạnh mẽ dùng để xây dựng giao diện người dùng dạng component. Hỗ trợ Virtual DOM giúp render nhanh và hiệu quả. Dễ mở rộng với các thư viện khác như React Router, Axios, Zustand.

1.4.2. Node.js + Express: Node.js là môi trường thực thi JavaScript phía server, non-blocking I/O giúp xử lý nhiều request cùng lúc. Express.js là framework nhẹ, giúp xây dựng RESTful API nhanh và dễ mở rộng. Hỗ trợ middleware (CORS, Multer, JWT) linh hoạt.

1.4.3. MySQL + MySQL2: Hệ quản trị cơ sở dữ liệu quan hệ phổ biến, hiệu suất cao. MySQL2 hỗ trợ Promise API, tối ưu tốc độ và bảo mật khi truy vấn. Dễ dàng tích hợp với Express để thao tác dữ liệu trong mô hình MVC.

1.4.4. JWT & Bcrypt: JWT là cơ chế xác thực người dùng an toàn, không cần lưu session. Bcrypt dùng để mã hóa mật khẩu, tăng tính bảo mật thông tin người dùng.

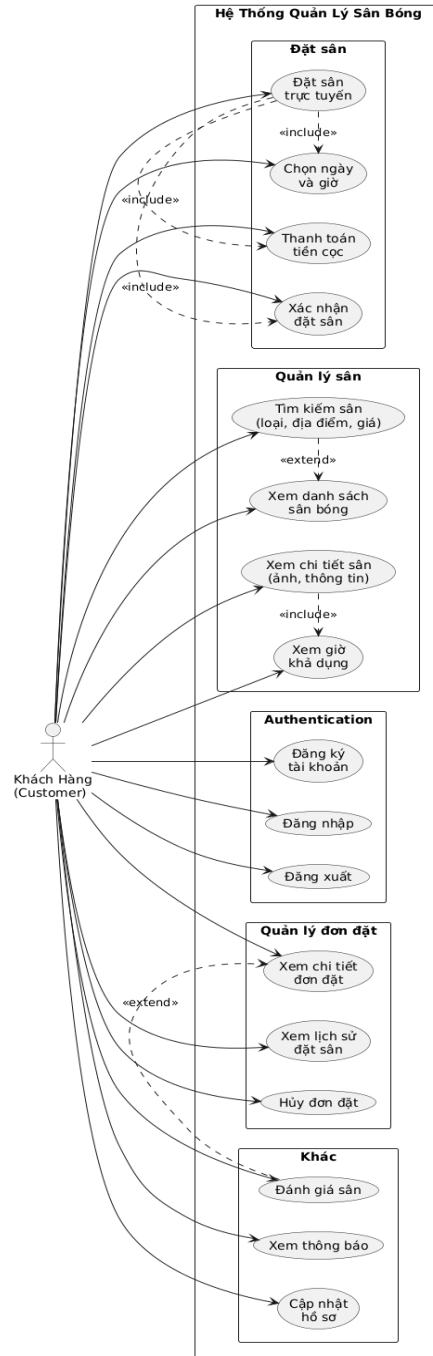
1.4.5. Multer: Middleware dùng để xử lý upload file (ảnh sản phẩm) trong Node.js, hỗ trợ nhiều định dạng.

1.4.6. Công cụ hỗ trợ: Postman, Git + GitHub, Nodemon, Dotenv.

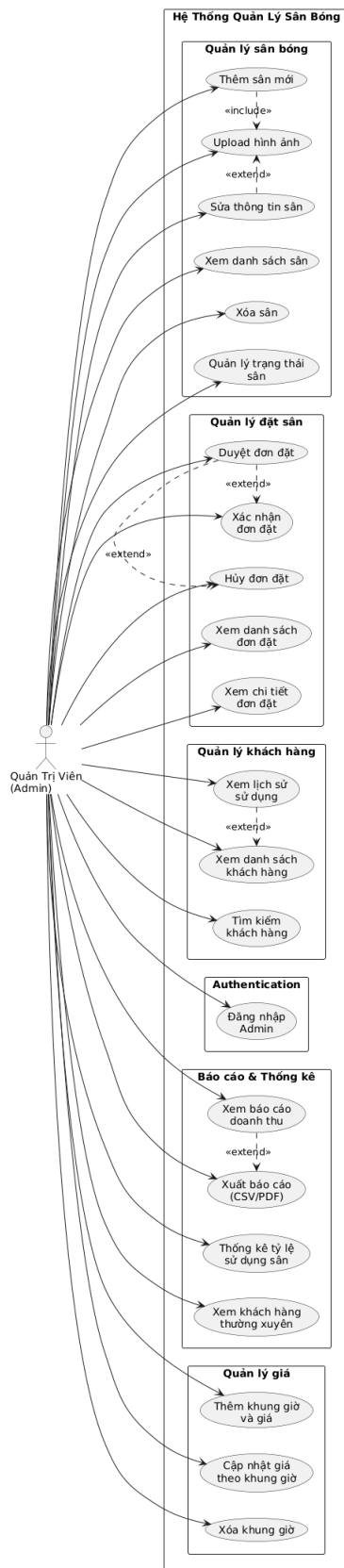
CHƯƠNG 2 : THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu hệ thống

Phân tích Use Case: Sơ đồ Use Case tổng quát cho thấy sự phân tách rõ ràng về quyền hạn:



Hình 1 : Use Case Diagram - Khách Hàng (Customer)



Hình 2 : Use Case Diagram - Quản Trị Viên (Admin)

2.1.1. Yêu cầu chức năng

Hệ thống quản lý sân bóng mini cần hỗ trợ đầy đủ quy trình nghiệp vụ từ khâu khách hàng tìm kiếm sân, đặt lịch, thanh toán đến khi quản trị viên theo dõi doanh thu và tình trạng khai thác sân.

Về phía khách hàng, hệ thống cho phép xem danh sách sân đang hoạt động, lọc theo loại sân, vị trí, khung giờ, giá thuê, xem chi tiết hình ảnh và tiện ích trước khi ra quyết định đặt sân.

Khách hàng có thể tạo tài khoản, đăng nhập, quản lý thông tin cá nhân, thực hiện đặt sân trực tuyến, xem lịch sử đặt sân và nhận thông báo khi đơn đặt được xác nhận hoặc bị hủy.

Về phía quản trị viên, hệ thống phải cung cấp các chức năng quản lý danh mục sân, thiết lập khung giờ và đơn giá, duyệt hoặc hủy các yêu cầu đặt sân, theo dõi lịch sử giao dịch và trích xuất báo cáo thống kê phục vụ quản lý kinh doanh.

2.1.2. Yêu cầu phi chức năng

Hệ thống cần đảm bảo hiệu năng đáp ứng tốt trong bối cảnh nhiều người dùng truy cập đồng thời trong các khung giờ cao điểm, đặc biệt khi thực hiện thao tác kiểm tra tình trạng trống và xác nhận đặt sân.

Thời gian phản hồi cho các thao tác tra cứu và đặt sân nên được giữ ở mức thấp, nhằm mang lại trải nghiệm mượt mà trên cả máy tính lẫn thiết bị di động.

Bảo mật là yêu cầu then chốt, trong đó thông tin đăng nhập và dữ liệu cá nhân của người dùng phải được mã hóa, phân quyền truy cập rõ ràng giữa khách hàng và quản trị viên, đồng thời hệ thống cần có cơ chế ngăn chặn truy cập trái phép và các lỗ hổng phổ biến như SQL Injection hay XSS.

Khả năng mở rộng cũng được đặt ra, để trong tương lai có thể dễ dàng bổ sung thêm chi nhánh sân mới, tích hợp cổng thanh toán trực tuyến hoặc các kênh thông báo như email, SMS mà không ảnh hưởng nhiều tới kiến trúc hiện tại.

2.1.3. Đối tượng sử dụng và ràng buộc

Đối tượng sử dụng hệ thống chủ yếu là nhân viên quản lý sân bóng và khách hàng phổ thông nên giao diện phải trực quan, dễ thao tác, hạn chế các bước phức tạp trong quá

trình đặt sân.

Bên cạnh đó, hệ thống được xây dựng trên nền tảng web nên cần tương thích với các trình duyệt phổ biến, hoạt động ổn định trên mạng Internet có băng thông trung bình, và hỗ trợ hiển thị tốt trên màn hình điện thoại kích thước nhỏ.

Các ràng buộc về công nghệ bao gồm việc sử dụng React cho tầng giao diện, Node.js và Express cho tầng nghiệp vụ, kết hợp MySQL làm cơ sở dữ liệu quan hệ, do đó thiết kế yêu cầu phải phù hợp với mô hình ba lớp và tận dụng ưu điểm của từng công nghệ này.

2.2. Kiến trúc hệ thống (System Architecture)

Hệ thống được thiết kế theo mô hình Kiến trúc phân tầng (Layered Architecture), cụ thể là kiến trúc 3 tầng (Three-tier Architecture). Việc chia tách này giúp hệ thống dễ bảo trì, dễ kiểm thử và có khả năng mở rộng độc lập từng thành phần.

Tầng 1: Presentation Layer (Tầng Giao diện - Frontend) Được xây dựng bằng ReactJS kết hợp với Bootstrap 5. Tầng này chịu trách nhiệm hiển thị dữ liệu cho người dùng và thu thập các tương tác. Các thành phần chính bao gồm:

- Components: Các khối giao diện tái sử dụng (Header, Footer, PitchCard, BookingForm).
- Pages: Các trang chức năng (HomePage, SearchPage, PitchDetailPage, AdminDashboard).
- State Management: Quản lý dữ liệu tạm thời của ứng dụng phía Client.

Tầng 2: Business Logic Layer (Tầng Nghiệp vụ - Backend) Được xây dựng trên nền tảng Node.js và Express. Đây là "bộ não" của hệ thống, xử lý mọi logic nghiệp vụ trước khi ghi xuống cơ sở dữ liệu. Luồng xử lý request tuân theo mẫu thiết kế MVC (Model - View – Controller).

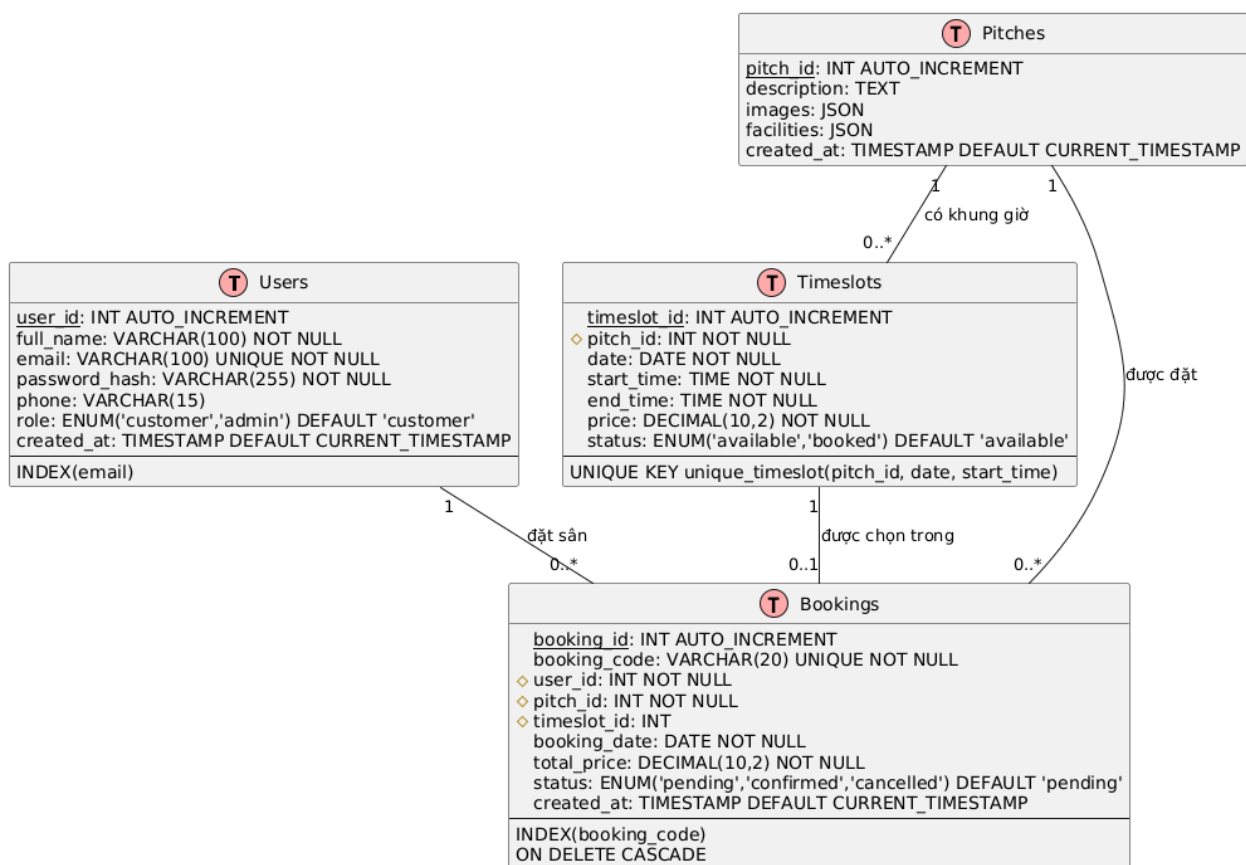
- Routes: Định nghĩa các điểm cuối (Endpoints) của API.
- Middleware: Thực hiện xác thực (Authentication), phân quyền (Authorization), và kiểm tra dữ liệu đầu vào (Validation).
- Controller: Chứa logic nghiệp vụ chính, điều phối việc gọi Model.

- Model: Tương tác trực tiếp với Database.[1, 1]

Tầng 3: Data Access Layer (Tầng Dữ liệu - Database) Sử dụng MySQL để lưu trữ bền vững. Tầng này bao gồm các bảng dữ liệu, Views, và Stored Procedures (nếu cần) để đảm bảo hiệu suất truy vấn.

2.3. Thiết kế Cơ sở dữ liệu (Database Design)

Cơ sở dữ liệu được thiết kế ở mức chuẩn hóa 3NF để giảm thiểu dư thừa dữ liệu. Dưới đây là phân tích chi tiết cấu trúc các bảng (Schema) và ý nghĩa của chúng trong việc giải quyết bài toán nghiệp vụ.



Hình 3: Sơ đồ quan hệ thực thể (ERD)

2.3.1. Bảng Users (Người dùng)

Bảng này lưu trữ thông tin định danh và xác thực.

```

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  phone VARCHAR(20),
  role ENUM('customer', 'admin') DEFAULT 'customer',
  avatar VARCHAR(500),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_email (email),
  INDEX idx_role (role)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Hình 4 : Cấu trúc bảng Users (Người dùng)

Phân tích: Việc sử dụng INDEX trên trường email là bắt buộc để tối ưu hóa tốc độ đăng nhập, vì đây là trường khóa tìm kiếm chính. Trường role sử dụng kiểu ENUM giúp giới hạn giá trị nhập vào, đảm bảo tính nhất quán của phân quyền ngay từ tầng database. Mật khẩu sẽ được lưu dưới dạng chuỗi băm (hash) chứ không phải văn bản thuần.

2.3.2. Bảng Pitches (Sân bóng)

Lưu trữ thông tin chi tiết về các sân bóng. Điểm đặc biệt trong thiết kế này là việc sử dụng kiểu dữ liệu JSON.

```

CREATE TABLE pitches (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  type ENUM('5v5', '7v7', '11v11') NOT NULL,
  location VARCHAR(500) NOT NULL,
  address TEXT,
  price_per_hour DECIMAL(10, 2) NOT NULL,
  description TEXT,
  images JSON,
  facilities JSON,
  status ENUM('active', 'maintenance', 'inactive') DEFAULT 'active',
  rating DECIMAL(3, 2) DEFAULT 0.00,
  total_reviews INT DEFAULT 0,
  opening_time TIME DEFAULT '06:00:00',
  closing_time TIME DEFAULT '23:00:00',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_type (type),
  INDEX idx_status (status),
  INDEX idx_rating (rating)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Hình 5 : Cấu trúc bảng Pitches (Sân bóng)

Phân tích: Việc sử dụng cột images và facilities kiểu JSON cho thấy sự linh hoạt trong thiết kế (Hybrid approach). Thay vì phải tạo bảng phụ pitch_images hay pitch_facilities và thực hiện các phép JOIN tốn kém, ta có thể lưu trữ danh sách ảnh và tiện ích ngay trong bảng pitches. Điều này giúp đơn giản hóa truy vấn khi hiển thị chi tiết sân mà không ảnh hưởng nhiều đến hiệu năng tìm kiếm, do MySQL hiện đại đã hỗ trợ đánh index trên trường JSON.

2.3.3. Bảng Timeslots (Khung giờ & Tính khả dụng)

Đây là bảng quan trọng nhất để xử lý logic "trống sân".

```
CREATE TABLE timeslots (
  id INT AUTO_INCREMENT PRIMARY KEY,
  pitch_id INT NOT NULL,
  date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  is_available BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (pitch_id) REFERENCES pitches(id) ON DELETE CASCADE,
  INDEX idx_pitch_date (pitch_id, date),
  INDEX idx_availability (is_available),
  UNIQUE KEY unique_timeslot (pitch_id, date, start_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Hình 6 : Cấu trúc bảng Timeslots (Khung giờ & Tính khả dụng)

Phân tích: Ràng buộc UNIQUE KEY unique_timeslot (pitch_id, date, start_time) đóng vai trò là chốt chặn bảo vệ cuối cùng của hệ thống. Ngay cả khi logic phần mềm bị lỗi, Database sẽ từ chối bất kỳ lệnh INSERT nào cố gắng tạo ra một khung giờ trùng lặp cho cùng một sân trong cùng một ngày. Đây là cơ chế ngăn chặn "Double Booking" hiệu quả nhất.

2.3.4. Bảng Bookings (Đơn đặt sân)

Lưu trữ thông tin giao dịch, liên kết người dùng với sân bóng tại một thời điểm cụ thể.

```

CREATE TABLE bookings (
  id INT AUTO_INCREMENT PRIMARY KEY,
  booking_code VARCHAR(50) UNIQUE NOT NULL,
  user_id INT NOT NULL,
  pitch_id INT NOT NULL,
  booking_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  duration INT NOT NULL,
  total_price DECIMAL(10, 2) NOT NULL,
  deposit_price DECIMAL(10, 2),
  status ENUM('pending', 'confirmed', 'cancelled', 'completed') DEFAULT 'pending',
  notes TEXT,
  cancellation_reason TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (pitch_id) REFERENCES pitches(id) ON DELETE CASCADE,
  INDEX idx_booking_code (booking_code),
  INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Hình 7 : Cấu trúc bảng bookings (Đơn đặt sân)

Phân tích: Trường booking_code được đánh index UNIQUE để phục vụ việc tra cứu đơn hàng nhanh chóng mà không cần lộ id tăng tự động (tránh việc đoán mò số lượng đơn hàng). Cấu trúc ON DELETE CASCADE đảm bảo rằng nếu một user hoặc một pitch bị xóa, tất cả dữ liệu booking liên quan sẽ được dọn dẹp tự động, giữ cho database luôn sạch.

CHƯƠNG 3: ĐÁNH GIÁ CÔNG NGHỆ VÀ HIỆN THỰC HỆ THỐNG

3.1. Đánh giá lựa chọn Frontend - React.js

Việc lựa chọn React.js cho tầng Frontend dựa trên các phân tích kỹ thuật sau:

Bảng 1 . Bảng đánh giá React theo tiêu chí

Tiêu chí	Đánh giá	Lý do chi tiết
Độ phức tạp	★ ★ ★ (3/5)	React đòi hỏi lập trình viên nắm vững khái niệm JSX, Hooks và quản lý State bất đồng bộ. Tuy nhiên, với nền tảng JavaScript vững chắc, việc tiếp cận mất khoảng 2-3 tuần.
Hiệu năng	★ ★ ★ ★ ★ (5/5)	Cơ chế Virtual DOM giúp giảm thiểu các thao tác vẽ lại (reflow/repaint) trên trình duyệt, mang lại trải nghiệm mượt mà ngay cả khi hiển thị danh sách hàng trăm sản phẩm.
Khả năng mở rộng	★ ★ ★ ★ ★ (5/5)	Kiến trúc dựa trên Component cho phép tái sử dụng code cao. Ví dụ: Component PitchCard được dùng ở cả Trang chủ, Trang tìm kiếm và Trang lịch sử.
Cộng đồng	★ ★ ★ ★ ★ (5/5)	Hệ sinh thái khổng lồ với hàng nghìn thư viện hỗ trợ sẵn.

Các thư viện hỗ trợ đã triển khai:

- React Router v6: Quản lý định tuyến phía Client, hỗ trợ Nested Routes cho các layout phức tạp như Dashboard Admin.
- React Hook Form: Xử lý các form đặt sản phẩm phức tạp với chi phí render thấp nhất, tích hợp validation dễ dàng.
- Axios: Client HTTP mạnh mẽ, hỗ trợ Interceptors để tự động đính kèm JWT Token vào header của mọi request gửi đi.

Rủi ro và Giải pháp:

Rủi ro lớn nhất của React là Bundle Size (kích thước file nén) có thể lớn, ảnh hưởng tốc độ tải trang đầu. Giải pháp được áp dụng là Code Splitting và Lazy Loading - chỉ tải code của trang Admin khi người dùng thực sự truy cập vào route /admin.

3.2. Đánh giá lựa chọn Backend - Node.js + Express.js

Node.js được chọn thay vì các nền tảng truyền thống như PHP hay Java Spring Boot vì tính đồng nhất ngôn ngữ (Full-stack JavaScript) và hiệu năng I/O.

Bảng 2 . Bảng đánh giá Node.js + Express.js theo tiêu chí kỹ thuật

Tiêu chí	Đánh giá	Ghi chú kỹ thuật
Hiệu năng	★★★★ (4/5)	Rất tốt cho các tác vụ I/O-bound (truy vấn DB, đọc ghi file) nhờ cơ chế Non-blocking. Tuy nhiên, cần lưu ý không thực hiện các tác vụ tính toán nặng (CPU-bound) trên luồng chính.
Khả năng mở rộng	★★★★ (4/5)	Hỗ trợ Horizontal Scaling thông qua Cluster Mode hoặc PM2, tận dụng tối đa số nhân CPU của server.
Bảo mật	★★★ (3/5)	Express mặc định khá "mở", do đó cần tích hợp thêm các middleware bảo mật như helmet, cors, và express-rate-limit.

Các gói thư viện (Packages) quan trọng:

- mysql2: Driver kết nối MySQL hỗ trợ Promise, giúp tránh tình trạng "Callback Hell" và viết code sạch hơn với async/await.
- jsonwebtoken (JWT): Tạo và xác thực token cho cơ chế đăng nhập không trạng thái.
- bcryptjs: Thư viện chuẩn công nghiệp để băm mật khẩu. Sử dụng Salt Rounds = 10 để cân bằng giữa độ bảo mật và hiệu năng.
- Multer: Middleware xử lý upload file multipart/form-data, dùng để upload ảnh sản phẩm vào thư mục local (hoặc cloud storage trong tương lai).

3.3. So sánh và lựa chọn Database (MySQL vs NoSQL)

Quyết định sử dụng MySQL thay vì MongoDB (thường thấy trong MERN Stack)

là một quyết định chiến lược dựa trên tính chất dữ liệu.

Bảng 3 . Đánh giá lựa chọn cơ sở dữ liệu MySQL

Database	Ưu điểm	Nhược điểm	Đánh giá phù hợp
MySQL	Hỗ trợ ACID, Transaction mạnh mẽ, tính toàn vẹn dữ liệu cao.	Khả năng mở rộng chiều ngang (Sharding) khó hơn NoSQL.	★★★★★ (5/5) - Tối ưu cho hệ thống Booking cần độ chính xác tuyệt đối.
MongoDB	Schema linh hoạt, mở rộng dễ dàng, tốc độ ghi cao.	Không hỗ trợ ACID đầy đủ ở các phiên bản cũ, mô hình Eventual Consistency có thể gây lỗi trùng lịch.	★★ (2/5) - Rủi ro cao về tính nhất quán dữ liệu đặt sân.

3.4. Cơ chế Bảo mật và Xác thực (Authentication & Security)

Hệ thống triển khai mô hình bảo mật nhiều lớp:

1. Xác thực (Authentication): Sử dụng JWT (JSON Web Token). Khi đăng nhập thành công, Server cấp một Token chứa thông tin user_id và role. Client lưu Token này (vào LocalStorage hoặc HttpOnly Cookie) và gửi kèm trong Header Authorization: Bearer <token> ở các request tiếp theo. Token có thời hạn (Expiration) là 7 ngày.
2. Phân quyền (Authorization): Middleware kiểm tra Role trong Token. Nếu user có role customer cố tình truy cập API /admin/pitches, hệ thống sẽ trả về lỗi 403 Forbidden.
3. Bảo vệ dữ liệu: Mật khẩu người dùng được băm bằng Bcrypt trước khi lưu. Ngay cả khi Database bị lộ, kẻ tấn công cũng không thể dịch ngược ra mật khẩu gốc.
4. Chống tấn công: Sử dụng express-validator để lọc dữ liệu đầu vào, ngăn

chặn SQL Injection và XSS Attacks.

CHƯƠNG 4: KẾ HOẠCH TRIỂN KHAI VÀ QUẢN LÝ DỰ ÁN

4.1. Phân tích hiện trạng và Tiến độ

Tại thời điểm báo cáo, dự án đã đi được khoảng 45% chặng đường phát triển tính năng. Các giai đoạn nền tảng như Phân tích yêu cầu, Thiết kế Hệ thống, Thiết kế Database và API Specification đã hoàn thành 100%. Giai đoạn phát triển (Development) đang diễn ra song song: Backend đạt 40% và Frontend đạt 30%.

Chi tiết các module đã hoàn thành:

- Phân tích và thiết kế kiến trúc toàn hệ thống.
- Xây dựng xong lược đồ CSDL (ERD) và khởi tạo trên MySQL.
- Định nghĩa xong API Endpoints (Input/Output).
- Hoàn thiện Module Authentication (Register/Login).
- Đang phát triển Module Booking và tích hợp UI cho trang chủ.

4.2. Lộ trình phát triển chi tiết (Sprint Planning)

Kế hoạch tập trung hoàn toàn vào việc phát triển và hoàn thiện các chức năng (Development) trong 4 Sprint chính, mỗi Sprint kéo dài khoảng 2 tuần, tuân thủ phương pháp Agile Scrum để đảm bảo tính linh hoạt.

Sprint 1 (Tuần 1-2): Backend Core

- Tập trung xây dựng nền móng.
- Nhiệm vụ: Thiết lập Server Express, kết nối MySQL, viết API Authentication và API CRUD cơ bản cho Sân bóng.

Kết quả: 15 API hoạt động, Postman Collection để test.

Sprint 2 (Tuần 3-4): Backend Advanced (Giai đoạn hiện tại)

- Xử lý các logic nghiệp vụ khó.
- Nhiệm vụ: Viết API Booking (xử lý transaction, check availability), Hệ thống thông báo (Notifications), Tích hợp Multer để upload ảnh.
- Kết quả: Full API Coverage, Swagger Documentation.

Sprint 3 (Tuần 5-6): Frontend Customer

- Bắt đầu ghép giao diện người dùng.
- Nhiệm vụ: Code trang Login/Register, Trang danh sách sân (kết nối API Search/Filter), Trang chi tiết sân.
- Kết quả: Giao diện phía khách hàng hoàn chỉnh, Responsive trên mobile.

Sprint 4 (Tuần 7-8): Frontend Admin

- Xây dựng trang quản trị.
- Nhiệm vụ: Dashboard thống kê (dùng Chart.js hoặc Recharts), Các trang quản lý (Table list, Form edit) cho Sân và Đơn hàng.
- Kết quả: Admin Panel đầy đủ chức năng, hỗ trợ Export báo cáo.